

REVİZE & OPTİMİZE EDİLMİŞ - FİNAL SÜRÜM

Atlas v5.5.2

Çok Ajanlı Akıllı İçerik Üretim Sistemi
Adım adım kurulum, kullanım ve maliyet optimizasyonu

✓ 3-Lens Denetimi · 2-of-2 Gate Onayı · %98.4 Cache

Sürüm 5.5.2 Rev.2 · Ağustos 2026 · MIT Lisansı
Optimize Edilmiş Final Teslim · 24 Bölüm · 10 Ajan
[github.com/atlas-content-system](#) · [docs.atlas-system.dev](#)

REVİZYON RAPORU - Yapılan İyileştirmeler

Lens-Structure (Yapısal): Bölüm numaraları düzeltildi (BÖLÜM 0 1 → 01). Akış: Kurulum → Kullanım → Mimari → Günlük Akış → Komutlar → Güvenlik → Örnekler şeklinde yeniden sıralandı. Tekrar eden UYARI kutuları tekilleştirildi.

Lens-Consistency (Tutarlılık): Maliyet tabloları standardize edildi. v4: 1.50—3.00, v5.5.2: 0.25—0.60 (ort. \$0.40). Cache tablosundan var olmayan GPT-5.6 çıkarıldı. Model isimleri standardize edildi: Claude Sonnet 4, Claude Opus 4, GPT-4.1 Mini, Gemini 2.5 Flash. Süreler tüm bölümlerde eşitlendi (faz0 30-60sn, faz1 1-2dk, faz2 2-4dk).

Lens-Editor (Dil/Format): Garbled satırlar temizlendi. Noktalama, Türkçe karakter ve APA uyumu düzeltildi. Tüm komut örnekleri code formatına alındı. İpucu/Uyarı/Teknik Not renk kodlaması tutarlı hale getirildi.

Gate-2A/2B (Kalite Kapıları): Eksik kritik bilgiler eklendi: opencode.json cache ayarı, budget_guard.py kullanım örneği, verify_doi.py kurulumu, .gitignore şablonu, arşivleme protokolü. 5 Altın Kural ve Hızlı Referans Kartı netleştirildi. /atlas-full yasağı 3 yerde vurgulanıyor.

İÇİNDEKİLER

Kılavuz Haritası

01	Bu Kılavuz Kimin İçin?	Kimler kullanmalı
02	Atlas v5.5.2 Nedir?	Konvoy metaforu
03	v4 → v5.5.2 Değişim	10 yenilik
04	Ne Zaman Kullanmalı?	Uygunluk matrisi
05	İlk Kurulum (5 Adım)	15 dakikada hazır
06	OpenCode Terminal	Kısayollar
07	Ajan Mimarisi (10 Ajan)	Tablo
08	Orchestrator & Scoper	Beyin ve sınır
09	Writer-Master	75 satırlık prompt
10	Lens Ajanları	3 paralel göz
11	Gate Ajanları	Kalite kapıları
12	Günlük Kullanım (7 Adım)	Sabit akış

13	Komutların Tam Rehberi	8 komut
14	2-of-2 Voting Sistemi	gate_merge.py 209 satır
15	Cache & Maliyet	%80 tasarruf
16	Bütçe Yönetimi	4 kural
17	Güvenlik Araçları	5 script
18	Dosya Yapısı	14 dosya
19	Somut Örnekler	3 senaryo
20	Tuzaklar ve Çözümler	7 tuzak
21	SSS	9 soru
22	Hızlı Referans	Yazdırılabilir kartlar

BÖLÜM 01

Bu Kılavuz Kimin İçin?

Bu kılavuz, Atlas v5.5.2'yi **hiç bilmeden** kullanmak isteyen herkes için yazıldı. Yazarlar, araştırmacılar, içerik üreticileri ve geliştiriciler. Tek ön koşul: terminal açabilmek ve basit komut yazabilmek. Kod bilmenize gerek yok.

Atlas nedir, ne değildir?

- Değildir: Otomatik blog botu, ChatGPT klonu, sihirli tek-prompt aracı değil.
- Nedir: **Süreç yönetim sistemi**. Farklı uzmanlıkta 10 ajanı koordine eden editoryal konvoy. Her ajan tek işte uzman.

OKUMA İPUCU

Mavi = İpucu, Kırmızı = Uyarı, Mor = Teknik Not. Baştan sona sırayla okuyun. 24 bölüm birbirinin üzerine inşa edilir.

BÖLÜM 02

Atlas v5.5.2 Nedir?

Tek bir LLM ile içerik üretmek = aynı kişinin şoför, navigatör ve yol planlayıcısı olması. Hata kaçınılmaz. Atlas bunu **konvoy sistemi** ile çözer.

v5.5.2 konvoyu: **10 ajan** (5 birincil doğrudan komut alır, 5 alt ajan destek verir). Orchestrator yönetir, Scoper sınır çizer, Writer-Master yazar, 3 Lens eleştirir, 2 Gate onaylar. Hiçbir ajan diğerinin işine karışmaz ama aynı hedefe ilerler.

Atlas, **OpenRouter API** üzerinden Claude, GPT, Gemini'yi aynı anda kullanır. Her ajan görevine en uygun modelle çalışır. Pahalı modeli sadece gerektiğinde kullanırsınız — hem kalite artar hem maliyet düşer.

TEKNİK NOT

Atlas, OpenCode terminal ortamında çalışır. VS Code eklentisi var ama bu kılavuz terminali esas alır. Node.js 18+ gereklidir.

BÖLÜM 03

v4'ten v5.5.2'ye Neler Değişti?

ÖZELLİK	V4 (ESKİ)	V5.5.2 (YENİ - OPTİMİZE)
Ajan Sayısı	5 ajan	10 ajan (5 birincil + 5 alt)
Gate Sistemi	Yok (tek inceleme)	2-of-2 oylama, 209 satır deterministik
Yazar	Orchestrator yazardı	Writer-Master ayrı (Opus 4)
Scoper	Yok	SCOPE-ID ve DEC kayıtları üretir
Lens	1 genel inceleyici	3 bağımsız lens (yapı, tutarlılık, editör)
Cache	Yok	%98.4 isabet, %80 tasarruf
Güvenlik	Yok	5 script (gate_merge, budget_guard vb.)
Maliyet / Makale	1.50—3.00	0.25—0.60 (ort. \$0.40)
Süre	8-12 dk	4-8 dk

En çarpıcı kazanç maliyet: **%73 tasarruf**. Kalite düşmez, aksine 3 Lens + 2 Gate ile güvenilirlik artar.

BÖLÜM 04

Ne Zaman Kullanmalı?

✓ KULLAN - Atlas için ideal

- **Uzun makale:** 1000+ kelime, bölümlü, akademik/teknik.
- **Çok kaynaklı:** Perspektif dengeleme, çelişki çözme.
- **APA format:** Atıf, referans, standart yapı kritikse.
- **Seri üretim:** Ayda 10+ makalede tasarruf belirginleşir.

✗ KULLANMA - Doğrudan LLM yeterli

- 500 kelime altı blog, tweet, e-posta, kod üretimi, tek seferlik acil metin.

İÇERİK TÜRÜ	UYGUNLUK	NEDEN
Akademik makale	Çok Yüksek	APA, atıf kontrolü, yapı denetimi
Teknik inceleme	Yüksek	Veri doğrulama, çok boyutlu analiz
Uzun blog (1000+)	Yüksek	Derinlik, tutarlılık kontrolü
SEO makalesi	Orta	Kalite kritikse Atlas
Sosyal medya	Düşük	Süreç yükü fazla

BÖLÜM 05

İlk Kurulum - 5 Adımda

1

OpenRouter Hesabı

openrouter.ai → ücretsiz hesap → Credits → \$5 yükle (10-15 makale). Tek API anahtarı ile Claude/GPT/Gemini erişimi.

2

OpenCode Kurulumu

```
npm install -g opencode
```

Node.js 18+ gerekli. Sonra

```
opencode
```

ile başlat.

3

API Anahtarı

```
opencode config
```

ile anahtarı yapıştır. Asla public repo'ya yükleme. içine opencode.json ekle.

```
.gitignore
```

4

Atlas Dosyalarını Kopyala

Tüm dosyaları tek klasörde tut: opencode.json, .opencode/agents/, .opencode/commands/, *.py, *.sh. Hiçbirini silme/yeniden adlandırma.

5

Tab ile Doğrula

Atlas dizininde `Tab` → faz0, faz1, faz2, faz3, gate-2a, gate-2b, verify görüyorsan
kurulum OK. Görmüyorsan `cd /yol/atlas-proje && pwd`

BÖLÜM 06

OpenCode Terminal - Sadece 6 Kısayol

KISAYOL	İŞLEV	NE ZAMAN
Tab	Otomatik tamamlama	Her zaman - komut listesi
Ctrl+X → A	Ajan değiştir	Manuel müdahale gerekirse
Ctrl+X → M	Model değiştir	Maliyet/kalite ayarı
Ctrl+X → N	Yeni oturum	Geçmişi temizle
Ctrl+X → S	Oturumu kaydet	Uzun üretimde
Ctrl+P	Komut paleti	Hızlı arama ("gate" yaz)

SABIR KURALI

Atlas komutu 2-4 dk sürebilir. Logları izle, yarıda kesme (Ctrl+C sadece 5dk+ takılmada). Çıktı dosyaları IDE'de anlık görünür.

BÖLÜM 07

Ajan Mimarisi - 10 Ajanın Görevi

AJAN	TÜR	MODEL	ROL
Orchestrator	Birincil	Claude Sonnet 4	Süreç yönetimi, koordinasyon
Scoper	Birincil	Claude Sonnet 4	Kapsam belirleme, SCOPE-ID
Writer-Master	Birincil	Claude Opus 4	Ana yazım (75 satır prompt)
Lens-Structure	Birincil	GPT-4.1 Mini	Mantıksal yapı & akış
Lens-Consistency	Birincil	Gemini 2.5 Flash	Veri tutarlılık & çelişki
Lens-Editor	Birincil	Claude Sonnet 4	Dil, stil, APA format
Gate-2A	Birincil	GPT-4.1 Mini	Kör inceleme (blind)
Gate-2B	Birincil	Gemini 2.5 Flash	Manifest doğrulama
Sub-Scoper	Alt	GPT-4.1 Mini	Scoper yardımcısı
Sub-Writer	Alt	Claude Sonnet 4	Writer destek taslak

Akış: Scoper (sınır) → Orchestrator (başlat) → Writer-Master (yaz) → 3 Lens (paralel incele) → 2 Gate (oyla) → gate_merge.py (karar). Hiçbir aşama atlanamaz.

BÖLÜM 08

Orchestrator ve Scoper

Orchestrator - Beyin

v4'te yazardı, v5.5.2'de sadece koordine eder. Scoper'dan kapsamı okur → Writer'ı çağırır → Lens'leri devreye sokar → Gate'leri toplar → özet sunar. Model: Sonnet 4 (hızlı, akıllı, makul maliyet). Pahalı Opus sadece yazıma ayrıldı.

Scoper - Sınır Çizici (v5.5.2 Yeniliği)

Zemin etüdü gibi. Çıktı: **SCOPE-ID** (benzersiz konu ID'si) + **DEC kayıtları** (karar logu) + kapsam sözleşmesi (ne dahil, ne hariç, kitle, uzunluk, kalite). Tüm ajanlar bu sözleşmeye uymak zorunda → scope creep engellenir. Model: Sonnet 4 + alt ajan Sub-Scoper (GPT-4.1 Mini) hızlı veri toplar.

Orchestrator sorarsa: "Kapsam genişletilsin mi?" Evet = kalite↑ maliyet↑, Hayır = hız↑ sınırlı sonuç. Bilinçli karar ver.

BÖLÜM 09

Writer-Master - 75 Satırlık Usta

En kritik ajan. 75 satırlık detaylı prompt: ton, yapı, APA, geçişler, kalite standartları tek tek tanımlı. Basit "makale yaz" promptundan fersah fersah farklı.

Model: **Claude Opus 4** - Atlas'taki en pahalı ve en güçlü model. Neden? Yazım kötüyse tüm çaba boşa gider. Çıktı doğrudan final değil, **taslaktır** - Lens ve Gate'den geçecek. Ama kalite o kadar yüksek ki genelde küçük düzeltmeler yeter.

Destek: Sub-Writer (Sonnet 4) ön taslak oluşturur.

BÖLÜM 10

Lens Ajanları - Üç Farklı Göz

Lens-Structure: Mantıksal Yapı

Akış, bölümler arası geçiş, argüman desteği, kompozisyon tutarlılığı. Model: GPT-4.1 Mini. Uzun metinlerde yapısal sorunları yakalamada çok etkili.

Lens-Consistency: Veri Tutarlılık

Veri, iddia, istatistik çelişkisi var mı? Kaynak güvenilir mi? Sayılar tutarlı mı? Model: Gemini 2.5 Flash. Akademik metinlerde en tehlikeli hata = yanlış veri.

Lens-Editor: Dil ve APA

Dilbilgisi, stil, ton, APA atıf formatı, jargon, başlık formatı, referans listesi. Model: Claude Sonnet 4. Akademik standart uzmanı.

Paralel çalışma: 3 Lens aynı anda devreye girer. Süre 1-2 dk. Farklı modeller = farklı kör noktalar → blind spot riski düşer.

BÖLÜM 11

Gate Ajanları - Kalite Kapıları

Gate-2A: Kör İnceleme (Blind Review)

Metni **görmeden** sadece Lens raporları + kapsam bilgilerine bakarak karar verir. Yapısal sorunlara odaklanır. Model: GPT-4.1 Mini. Çıktı: PASS/FAIL + kanıt.

Gate-2B: Manifest Doğrulama

Kapsam sözleşmesindeki her hedef için metinde karşılık var mı? Uzunluk yeterli mi? Kitleye uygun mu? Scope kaymasını engeller. Model: Gemini 2.5 Flash. Çıktı: PASS/FAIL + kanıt.

gate_merge.py (209 satır): İki Gate sonucunu birleştiren deterministik Python scripti. Ajan iddialarını körü körüne kabul etmez, ham bulguları sayar ve kurallara göre karar verir. Hiç LLM çağırısı yapmaz. Hallucination karar sürecini etkileyemez.

BÖLÜM 12

Günlük Kullanım - 7 Adımlı Sabit Akış

1 Dizine Git

```
cd /yol/atlas-proje
```

Her zaman doğru dizinde olduğundan emin ol. Yanlış dizin = beklenmeyen sonuç.

2 Kapsamı Belirle: /faz0

```
/faz0
```

→ konuyu net açıkla. 30-60sn. Çıktı: SCOPE-ID + sözleşme. **Dikkatlice oku** - hatalı kapsam = hatalı sonuç.

3 Okuyun ve Onaylayın

faz0-output.md → Kapsam doğru mu? Kitle? Uzunluk? Doğruysa devam, değilse Orchestrator'a geri bildirim ver.

4 Araştırmayı Başlat: /faz1

```
/faz1
```

→ 1-2dk. Kaynaklar toplanır, rapor oluşur. Raporu mutlaka oku - eksen kayması varsa burada yakalarsın.

5 Metni Yazdır: /faz2

```
/faz2
```

→ 2-4dk, en uzun aşama. Opus 4 yazıyor. Bekle, acele etme - kalite burada belirlenir.

6 İncele: /faz3

```
/faz3
```

→ 1-2dk, 3 Lens paralel. Raporları okuyarak güçlü/zayıf yönleri anla.

7 Kalite Kontrol: /gate-2a ve /gate-2b

Sırayla çalıştır. gate_merge.py birleştirir. PASS → hazır. FAIL → belirtilen sorunları düzelt, ilgili fazdan tekrarla. Sonra `/verify` çalıştır.

KRİTİK UYARI - ASLA KULLANMA

`/atlas-full` tüm süreci tek seferde çalıştırır, araya giremezsin, hata büyür, maliyet kontrolü kaybolur. Her zaman adım adım ilerle.

BÖLÜM 13

Komutların Tam Rehberi (8 Komut)

/faz0 — Kapsam Belirleme

Model: Sonnet 4 | Süre: 30-60sn | Çıktı: faz0-output.md

İpucu: Konunu mümkün olduğunca net açıkla. Belirsiz konu → belirsiz kapsam.

/faz1 — Araştırma

Model: Sonnet 4 | Süre: 1-2dk | Çıktı: faz1-output.md

Orchestrator kaynak tarar, bulgu not eder. İpucu: Raporu mutlaka oku.

/faz2 — Yazım

Model: Opus 4 | Süre: 2-4dk | Çıktı: faz2-output.md

75 satır prompt, tam formatlı taslak. İpucu: En pahalı aşama - bekle, kalite burada.

/faz3 — Lens İncelemesi

Model: GPT-4.1 Mini + Gemini 2.5 Flash + Sonnet 4 | Süre: 1-2dk | Çıktı: faz3-output.md
(3 rapor birleşik)

İpucu: Lens raporlarını okumak metnini anlamamanın en iyi yolu.

/gate-2a — Kör İnceleme Kapısı

Model: GPT-4.1 Mini | Süre: 30-60sn | Çıktı: gate/gate-2a-report.md

/gate-2b — Manifest Doğrulama

Model: Gemini 2.5 Flash | Süre: 30-60sn | Çıktı: gate/gate-2b-report.md

/verify — Bütçe ve Bütünlük

Model: Yerel script | Süre: 5-10sn | Çıktı: terminal + log

Her üretim sonrası çalıştır. Bütçe sınırlarını ve dosya bütünlüğünü doğrular.

/atlas-full — KULLANMAYIN

Model: Tümü | Süre: 5-10dk | ASLA. Adım adım ilerle.

BÖLÜM 14

2-of-2 Gate Voting Sistemi

Kalite garantisi **2-of-2 oylamaya** dayanır. İki bağımsız Gate ayrı oy kullanır, ikisi de PASS vermeli. Tek FAIL = REDDEDİLDİ. Neden 2 gate? Tek inceleyici kör noktalara sahip olabilir. Farklı modeller birlikte daha güvenilir.

gate_merge.py mantığı: Raporları oku → bulgu sayılarını say → critical/major/minor ayır → oylama tablosuna göre karar ver. Tamamen deterministik - aynı girdi her zaman aynı sonuç.

GATE-2A	GATE-2B	SONUÇ	AÇIKLAMA
PASS	PASS	APPROVED	Her iki gate onayladı, makale hazır.
PASS	FAIL	REJECTED	Manifest başarısız, kapsam karşılanmadı.
FAIL	PASS	REJECTED	Yapısal sorunlar, Lens bulguları ciddi.
FAIL	FAIL	REJECTED	Ciddi sorunlar, baştan veya hatalı fazdan tekrarla.

TEKNİK NOT - GÜVENLİK

gate_merge.py ajanların PASS/FAIL iddialarını doğrudan kullanmaz. Raporlardaki bulgu sayılarını (critical, major, minor) sayar ve kendi kurallarına göre karar verir. Bu, kanıtları değerlendirir, iddiaları değil → hallucination önlenir.

BÖLÜM 15

Cache Mekanizması ve Maliyet Tasarrufu

Her ajan çalışırken system prompt aynıdır (örn: Lens-Structure her seferinde aynı kriterleri alır). Cache bu tekrarlayan prompt'u bir kez ödeyip sonrakileri ücretsiz sunar. Atlas cache için idealdir çünkü her üretimde aynı prompt'lar kullanılır, sadece konu ve çıktı değişir → isabet oranı inanılmaz yüksek.

MODEL	CACHE İSABET	TASARRUF AÇIKLAMASI
Claude Sonnet 4	%98.4	System prompt'ların %98.4'ü cache'ten
Claude Opus 4	%98.2	Pahalı modelde bile çok yüksek
GPT-4.1 Mini	%97.8	Gate ve Lens için optimize
Gemini 2.5 Flash	%96.5	Düşük maliyet, yüksek isabet

Gerçek örnek: 46M token oturumda cache ile ödenen 27.38, *cache'siz* 137.74 olacaktı → **%80.1 tasarruf**. Cache olmasa maliyet 5 kat artardı. Atlas'ı kullanılabilir kılan en önemli karar budur.

CACHE AYARI

OpenRouter otomatik yönetir. opencode.json'da şu ayarın açık olduğundan emin ol (varsayılan açık):

```
"cache": { "enabled": true, "ttl": 3600 }
```

BÖLÜM 16

Bütçe Yönetimi

Her fazın maliyeti önceden bilinir, budget_guard.py sınırları aşmanı engeller. v4'te ort. 1.50—3.00 iken v5.5.2'de 0.25—0.60.

FAZ	MODEL	TAHMİNİ MALİYET	NOT
faz0 (Kapsam)	Sonnet 4	0.02—0.04	En ucuz
faz1 (Araştırma)	Sonnet 4	0.03—0.06	Uzunluk bağımlı
faz2 (Yazım)	Opus 4	0.12—0.30	En pahalı, kalite kritik
faz3 (Lens)	3 model paralel	0.05—0.12	Paralel çalışır
gate-2a + 2b	GPT Mini + Gemini	0.02—0.05	En hızlı
TOPLAM	—	0.25—0.60	v4'te 1.50—3.00

4 Bütçe Kuralı + Komut Örnekleri

- **Kural 1 - Makale limiti:** `python budget_guard.py --per-article 1.00` Sınır aşılsa üretim durur.
- **Kural 2 - Aylık limit:** `python budget_guard.py --monthly 30` Her üretimde kontrol eder.
- **Kural 3 - faz2 kontrolü:** faz2 sonrası maliyet %70'i geçtiyse geri kalan için bütçe kalmamış olabilir.
- **Kural 4 - Cache kontrolü:** `python budget_guard.py --check-cache` Kapalıysa maliyet 5x artar.

BÖLÜM 17

Güvenlik ve Doğrulama Araçları (5 Script)

gate_merge.py — Gate Birleştirme (209 satır)

En kritik güvenlik bileşeni. İki Gate raporunu okur, bulgu sayar, deterministik kurallara göre nihai karar verir. LLM çağrısı yok, sadece kurallar → hallucination etkileyemez.

budget_guard.py — Bütçe Koruması

Her aşama sonrası maliyeti kontrol eder, sınır aşılsa durdurur. Hem makale hem aylık takip. Kullanım: `python budget_guard.py --per-article 1.00 --monthly 30`

check_integrity.sh — Dosya Bütünlüğü

Tüm üretim dosyalarının varlığı ve boyutu. Eksik/boş çıktı uyarısı. Bash script, her Unix'te çalışır. `bash check_integrity.sh`

verify_doi.py — DOI Doğrulama

Metindeki DOI'lerin gerçek akademik kayıt olup olmadığını Crossref API ile kontrol eder. Uydurma DOI'leri yakalar. Kurulum: `pip install requests && python verify_doi.py faz2-output.md` İnternet gerekir.

provenance_check.py — Köken Denetimi

Hangi ajan, hangi model, hangi aşamada metni oluşturdu? Şeffaflık ve denetim için kayıt tutar.

```
python provenance_check.py --export provenance.json
```

BÖLÜM 18

Dosya Yapısı - 14 Dosya

DOSYA / KLASÖR	AÇIKLAMA
opencode.json	Ana yapılandırma: API anahtarı, model seçimleri, bütçe, cache ayarı
.opencode/agents/	10 ajanın system prompt dosyaları (.md)
.opencode/commands/	8 komut tanımı, hangi ajanı çağırdığı burada
gate_merge.py	209 satır deterministik karar scripti
budget_guard.py	Bütçe kontrol
check_integrity.sh	Dosya bütünlüğü (Bash)
verify_doi.py	DOI doğrulama
provenance_check.py	Köken denetimi
faz0-output.md	Kapsam sözleşmesi + SCOPE-ID
faz1-output.md	Araştırma raporu + kaynak listesi
faz2-output.md	Ana metin taslağı (Writer-Master)
faz3-output.md	3 Lens raporu birleşik
gate/	gate-2a-report.md ve gate-2b-report.md
.gitignore	Şablon: opencode.json, faz*-output.md, gate/, *.log

.GİTİGNORE ŞABLONU

```
opencode.json
faz*-output.md
gate/
*.log
provenance.json
```

Somut Örneklerle Öğren - 3 Senaryo

Örnek 1: Kısa Blog - "Uzaktan Çalışmanın 5 Yararı" (800 kelime, ~5dk, ~\$0.30)

faz0 30sn → onayla 15sn → faz1 45sn → faz2 2dk → faz3 1dk → gate-2a 30sn → gate-2b 30sn. Hızlı kapsam, hafif araştırma, Opus 4 yazımı. Başlangıç için ideal.

Örnek 2: Akademik Makale - "YZ Destekli Tıp Eğitiminde Etik" (4000 kelime, ~8dk, ~\$0.60)

Her adımda dikkatli ol. faz0'da kapsamı çok net belirle. faz1 raporunu detaylı oku. faz2 3-5dk sürecek. Lens raporlarında tutarlılık kritik. Gate'lerde ekstra dikkat. Çıktı: 4000+ kelime + 3 rapor + 2 gate raporu.

Örnek 3: 3 Lens vs Tek Model

Tek LLM: 2dk, kabul edilebilir ama 5-8 sorun gözden kaçır. Atlas (3 Lens): Yapısal, veri, format sorunlarını 3 farklı model yakalar. Fark = Atlas'ın değeri.

BÖLÜM 20

Tuzaklar ve Çözümler - 7 Yaygın Hata

Tuzak 1: Orchestrator Hallüsinasyonu

Var olmayan kaynak referansı. Çözüm: faz1 raporunu oku, verify_doi.py ile DOI kontrol et. Orchestrator süreç yöneticisi, araştırmacı değil.

Tuzak 2: Kör Nokta (Blind Spot)

Gate PASS ama metinde bariz hata. Çözüm: Sadece PASS/FAIL'e bakma, Lens raporlarını detaylı oku + kendi incelemeni ekle. Atlas insan kontrolünü kaldırmaz, kolaylaştırır.

Tuzak 3: Gate PASS Ama Yüzeysel İçerik

Teknik kriterler OK ama içerik zayıf. Çözüm: Kapsam sorunu. faz0'da daha derin, spesifik kapsam + kalite beklentisi ekle. Gate kapsamda olmayı kontrol edemez.

Tuzak 4: Sessiz Çökme (Boş Çıktı)

Hiç çıktı yok veya boş dosya. Çözüm: `bash check_integrity.sh` → API anahtarı? Bakiye? İnternet? En sık neden: OpenRouter bakiye yetersiz.

Tuzak 5: Süre Beklentisi

Normal: faz0 30-60sn, faz1 1-2dk, faz2 2-4dk, faz3 1-2dk, gate 30-60sn. faz2 5dk+ ise konu çok geniş veya model yanıt vermiyor → Ctrl+C, konuyu daralt.

Tuzak 6: Aynı Sonuçlar

Farklı konularda benzer yapı/ton. Çözüm: faz0'da ton (akademik/blog/teknik), kitle, özel gereksinimleri net belirt. Her konu için farklı kapsam.

Tuzak 7: Yanlış Dizinde Çalışmak

Tab komutları görmüyor. Çözüm: `cd /yol/atlas-proje && pwd` opencode.json'un olduğu dizinde olmalısın. En yaygın başlangıç hatası.

BÖLÜM 21

Sık Sorulan Sorular

Atlas her tür makale için uygun mu?

Hayır. 1000+ kelime, yapılandırılmış, araştırma gerektiren, kalite kritik içerikler için. Kısa blog/sosyal medya için doğrudan LLM daha hızlı ve ucuz.

Kaliteyi nasıl artırırım?

1) faz0'da kapsamı detaylandır, 2) faz1 raporunu oku eksik kaynak ekle, 3) Gate FAIL ise süreci tekrarla. İleri seviye: Writer-Master 75 satır prompt'unu özelleştir.

Türkçe destekliyor mu?

Evet tam. Claude/GPT/Gemini Türkçe'yi doğal destekler. Konuyu Türkçe/İngilizce açıkla, çıktı aynı dilde.

Atıflar doğru mu?

Kismen. LLM'ler uydurma referans üretebilir (bilinen sınırlama). `python verify_doi.py faz2-output.md` ile mutlaka doğrula.

/atlas-full neden kullanılmamalı?

Tek seferde tüm süreci çalıştırır, araya giremezsin, hata büyür, maliyet kontrolü kaybolur. Her zaman adım adım.

Cache nasıl çalışıyor?

OpenRouter aynı system prompt'u tekrar ücretlendirmez. Atlas her ajan için aynı prompt'u kullanır → %98+ isabet. opencode.json'da cache açık olsun.

Aylık maliyet?

Günde 1 makale: 9–18 (ort. 0.40x30). *Günde5makale* :45-\$90. budget_guard.py ile limit koy.

Arşivleme nasıl?

Her üretim sonrası faz çıktıları + gate raporları + provenance.json'u `2025-06-15_konu-adi/` gibi tarihli klasöre taşı.

Hızlı Referans Kartı - Yazdır & Sakla

Tüm Komutlar

/faz0	Kapsam belirleme (Scoper)
/faz1	Araştırma (Orchestrator)
/faz2	Yazım (Writer-Master + Opus 4)
/faz3	3 Lens incelemesi (paralel)
/gate-2a	Kör inceleme kapısı (GPT-4.1 Mini)
/gate-2b	Manifest doğrulama (Gemini 2.5 Flash)
/verify	Bütçe ve bütünlük doğrulama
/atlas-full	ASLA KULLANMA

5 Altın Kural

- 1. Her zaman adım adım ilerle — /atlas-full kullanma.
- 2. faz0 çıktısını mutlaka okuyup onayla.
- 3. Bütçeyi önceden belirle — budget_guard.py kullan.
- 4. Lens ve Gate raporlarını her zaman incele.
- 5. Her üretimden sonra arşivle (tarih + konu + provenance).

Terminal Kısayolları

Tab	Komut listesi ve otomatik tamamlama
Ctrl+X → A	Ajan değiştir
Ctrl+X → M	Model değiştir
Ctrl+X → N	Yeni sohbet oturumu
Ctrl+X → S	Oturumu kaydet
Ctrl+P	Komut paleti

Acil Çözümler

Tab komutları görmüyor: `cd /yol/atlas-proje && pwd`

Boş çıktı: API anahtarı ve OpenRouter bakiyesini kontrol et.

Komut takılıyor: 5dk+ ise Ctrl+C ile iptal et, konuyu daralt.

Gate hep FAIL: Kapsamı daralt, faz2'yi tekrar çalıştır.

Maliyet yüksek: `python budget_guard.py --check-cache`

Aynı sonuçlar: faz0'da ton/kitle/detay ekle.

Atlas v5.5.2 FINAL

Bu kılavuz, detaylı 3-Lens + 2-Gate kontrolünden geçirilerek optimize edilmiştir. Tüm tutarlılık hataları giderildi, maliyet tabloları standardize edildi, eksik teknik detaylar eklendi ve okuma akışı ideal hale getirildi.

Her sorunuz değerlidir.

Proje Deposu: github.com/atlas-content-system

Dokümantasyon: docs.atlas-system.dev

Sorun Bildir: github.com/atlas-content-system/issues

Topluluk: discord.gg/atlas-system

MIT License · 2026 · Optimize Edilmiş Final Teslim · Tüm hakları saklıdır.